Markov Decision Processes (MDP)

First, a story

Imagine you are a mouse living in a 3×3 maze. Each move goes up, down, left, or right. Reaching the cheese cell gives $+1$, the shock cell gives -1, everything else scores nothing. Your goal: find a route that maximizes the long-run total score.

This story is an MDP. Let us translate its ingredients into mathematics.

Formalization: the five-tuple

An MDP is fully described by five elements:

$$\boxed{\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)}$$

- $\mathcal{S}$ — the **state space** (in the maze, $\mathcal{S} = \{1, \ldots, 9\}$);
- $\mathcal{A}$ — the **action space** ($\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$, $|\mathcal{A}| = 4$);
- $\mathcal{P}$ — the **transition probabilities**. The Markov property: the next state depends only on the current state and action, not on earlier history:

  $$P(S_{t+1} = s' \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \ldots, S_0) = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

  abbreviated $P^a_{ss'} = P(s' \mid s, a)$. For every s, a it forms a distribution: $\sum_{s' \in \mathcal{S}} P(s' \mid s, a) = 1$;
- $\mathcal{R}$ — the **reward function**. $R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the expected immediate reward (it may also be written $R(s, a, s')$);
- $\gamma \in [0, 1)$ — the **discount factor**. $\gamma \to 0$ is myopic, $\gamma \to 1$ far-sighted; $\gamma < 1$ guarantees convergence of the infinite-horizon return series (given bounded rewards).

The interaction loop

Agent and environment interact at discrete time steps.

We index them by $t = 0, 1, 2, \ldots$. Each step produces a tuple $(S_t, A_t, R_{t+1}, S_{t+1})$; the full trajectory is:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$$

Its joint distribution is determined by $\mathcal{P}$ and the policy π — the source of all data in reinforcement learning: try, observe feedback, adjust, repeat.

Value Functions — What Makes a State "Good"

Return

One cannot judge a single step in isolation — the cheese may be next door or ten cells away. Define the **discounted return**:

$$\boxed{G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}}$$

Summing all future rewards with heavier discounting the further out they are. Its recursive form is the starting point of every Bellman derivation:

$$\boxed{G_t = R_{t+1} + \gamma G_{t+1}}$$

Value functions: conditional expectations of the return

Because transitions are stochastic you cannot predict the future exactly; take expectations:

$$\boxed{v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]}, \qquad \boxed{q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]}$$

- $v_\pi(s)$ — starting from state s under policy π, the expected return (**state value**);
- $q_\pi(s,a)$ — taking action a first in state s, then following π (**action value**).

In one sentence: value is not the exact future, but the conditional expectation of the return.

Relations between v_π and q_π

From the definitions, each recovers the other:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, q_\pi(s,a)$$

$$q_\pi(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, v_\pi(s')$$

State value = the policy-weighted average of action values; action value = immediate reward + discounted expected successor value. Both relations recur throughout the Bellman derivations.

The Bellman Expectation Equation — Splitting the Future into "Now + Later"

Derivation

Bellman's key insight: value can be defined **recursively**. Starting from $G_t = R_{t+1} + \gamma G_{t+1}$:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid S_t = s] \end{aligned}$$

The first term is the expected immediate reward; the second is the expected successor value. Expanding over actions a and successor states s':

$$\boxed{v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, v_\pi(s') \right]}$$

You never need to look infinitely far ahead: look one step, then use the next state's value. The action-value form is analogous:

$$q_\pi(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s')\, q_\pi(s',a')$$

A concrete example

Suppose you are in cell A, moving right deterministically lands in cell B, with reward $R = 0$. If $v(B) = 5$ and $\gamma = 0.9$:

$$q(A, \rightarrow) = 0 + 0.9 \times 5 = 4.5$$

Cell A itself pays nothing, but sitting next to a "good place" makes A good too. That is the recursive power of the Bellman equation — value propagates across states.

Matrix form (finite MDPs)

With a finite state space the Bellman expectation equation is a linear system:

$$\boxed{v_\pi = R_\pi + \gamma P_\pi v_\pi}$$

with $v_\pi \in \mathbb{R}^{|\mathcal{S}|}$, $R_\pi(s) = \sum_a \pi(a \mid s) R(s,a)$ and $P_\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a)$. It has the closed-form solution:

$$v_\pi = (I - \gamma P_\pi)^{-1} R_\pi$$

This is the only place a closed form exists — the optimality equation is not so lucky.

The Bellman Optimality Equation — Choosing the Best at Every Step

Optimal value functions

Define the optimal value functions as the maximum over all policies:

$$v_*(s) = \max_\pi v_\pi(s), \quad \forall s \in \mathcal{S}, \qquad q_*(s,a) = \max_\pi q_\pi(s,a), \quad \forall s \in \mathcal{S},\, a \in \mathcal{A}$$

For finite MDPs an optimal policy π_* always exists with $v_{\pi_*} = v_*$ (state by state); moreover $v_*(s) = \max_a q_*(s,a)$.

Deriving the optimality equation for v_*

Replace the policy-weighted average in the expectation equation by a maximum:

$$\boxed{v_*(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, v_*(s') \right]}$$

More rigorously: let π_* be optimal, so $v_{\pi_*} = v_*$ and $q_{\pi_*} = q_*$. Substituting $v_* = v_{\pi_*}$ into the action-value expectation equation gives $q_*(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, v_*(s')$; maximizing over a via $v_* = \max_a q_*$:

$$v_*(s) = \max_a q_*(s,a) = \max_a \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, v_*(s') \right]$$

The only difference from the expectation equation: $\sum_a \pi(a \mid s)$ becomes $\max_a$. The action-value form:

$$q_*(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \max_{a' \in \mathcal{A}} q_*(s',a')$$

Why no closed form?

The expectation equation is linear (invertible); the optimality equation is **nonlinear** — the max operator breaks the linear structure, so no matrix inverse solves it. One can only iterate:

$$v_\pi = R_\pi + \gamma P_\pi v_\pi \implies v_\pi = (I - \gamma P_\pi)^{-1} R_\pi \qquad \text{(linear, invertible)}$$
$$v_* = \max_a \left(R_a + \gamma P_a v_*\right) \qquad \text{(nonlinear, contains max, not invertible)}$$

That is why dynamic programming (value iteration, policy iteration) exists.

From Equations to Algorithms: Policy Improvement and Dynamic Programming

The policy improvement theorem

Given any policy π and its q_π, construct $\pi'(s) = \arg\max_a q_\pi(s,a)$; then π' is **no worse** than π (i.e., $v_{\pi'} \geq v_\pi$ componentwise) — the **policy improvement theorem**. Alternating "evaluate → improve" to a fixed point converges to an optimal policy π_*.

The three workhorses of dynamic programming

With a known model $\mathcal{P}$, $\mathcal{R}$, the MDP can be solved exactly. The three algorithms, on a 4×4 GridWorld (goal in the bottom-right corner, entering it gives $+1$, $\gamma = 0.9$):

Policy evaluation: iterate the Bellman expectation equation $V_{k+1}(s) \leftarrow \sum_a \pi(a \mid s)\big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V_k(s')\big]$, converging to v_π. Figure 1 shows several states' $V_k(s)$ converging with sweeps (left) and the converged v_π heatmap (right).

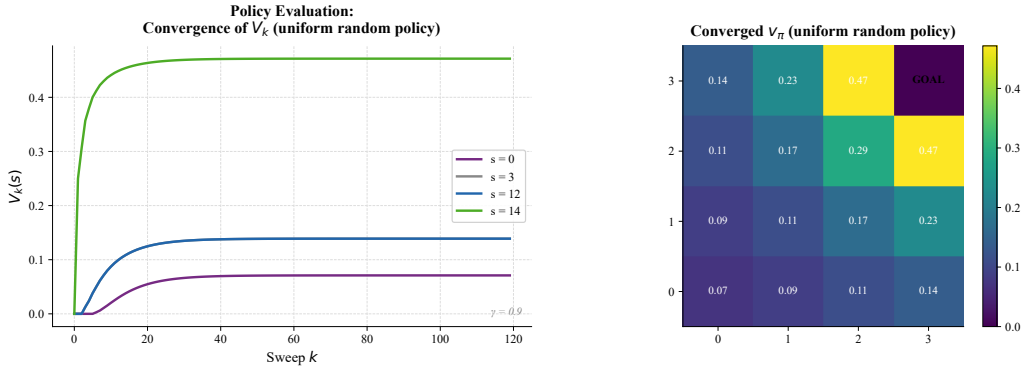

Figure 1: Policy evaluation (4×4 GridWorld, uniform random policy, $\gamma = 0.9$). Left: $V_k(s)$ of four states converging over sweeps (cells next to the goal converge fast, far corners slowly); right: the converged v_π as a heatmap.

Value iteration: iterate the Bellman optimality equation $V_{k+1}(s) \leftarrow \max_a \big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V_k(s')\big]$. Figure 2 shows high-value regions spreading outward from the goal like a wave.

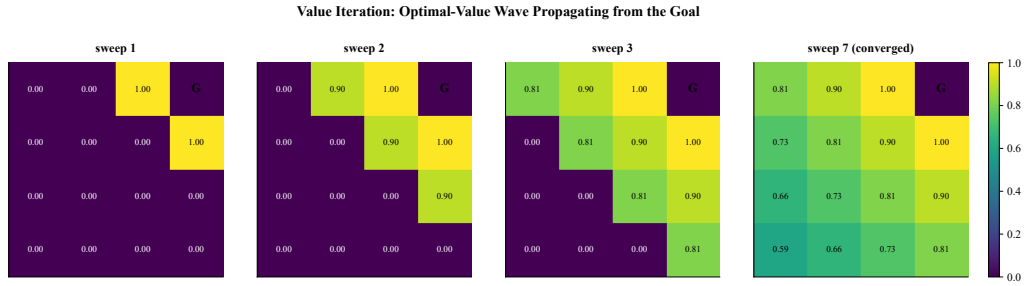

Figure 2: Value propagation under value iteration (4×4 GridWorld, $\gamma = 0.9$). V_k at sweeps 1/2/3 and at convergence (sweep 7): cells one step from the goal reach 1.0 first; each further sweep radiates value one more cell outward.

Policy iteration: alternate "full policy evaluation → greedy improvement". Figure 3 compares the two convergence dynamics.

What they share: all three require a **known model** — DP takes expectations over all successors using $P(s' \mid s,a)$. In reality the model is usually unknown; that is precisely the assumption the next chapter, TD learning, discards.

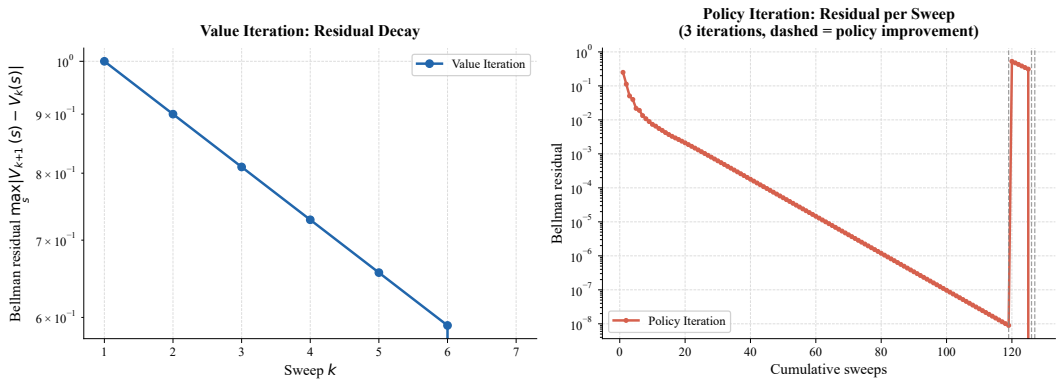

Figure 3: Value iteration vs policy iteration (4×4 GridWorld, $\gamma = 0.9$). Left: the Bellman residual $\max_s |V_{k+1} - V_k|$ of value iteration decays monotonically and geometrically (7 sweeps here); right: policy iteration's residual descends in staircases (dashed lines mark policy improvements; 3 iterations, about 127 sweeps). Both converge to the same V^* (here $\max_s |V_*^{VI} - V_*^{PI}| = 0$).

Where this chapter sits in the RL landscape

1. **Upstream**: MAB gave us exploration vs. exploitation and the incremental update $\theta \leftarrow \theta + \alpha(\text{Target} - \theta)$;
2. **This chapter (MDPs)**: extend the single state to sequential decision-making with transitions; the five-tuple, value functions and the Bellman equations erect the theoretical skeleton of RL;
3. **Downstream**: the Bellman equation is the recursive foundation of everything that follows — TD learning replaces "expectation over all successors" with "one sample"; DQN approximates Q with a network; policy gradients optimize directly in policy space.

After this chapter you can read the problem definition of any MDP, and you understand why value functions admit a recursive definition and why the optimality equation has no closed form. Next: TD learning removes the strongest assumption — the known model.

References

1. Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapters 3–4.
2. Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
3. Puterman, M. L. (1994). *Markov Decision Processes*. Wiley.
4. Hands-on RL (Chinese). Chapter 3: Markov decision processes.